Razonamiento Automático

RAG para lógica e Isabelle, con verificación formal

Ciencias de la Computación — Isabelle2025-2

Construir un asistente que responda preguntas sobre lógica, demostración automática y formalización en Isabelle/Isar, apoyándose en los documentos del curso, y que, cuando proponga una demostración, se la haga verificar a Isabelle antes de mostrarla.

El modelo propone; Isabelle decide.

1. Por qué este proyecto

Un modelo de lenguaje escribe con fluidez, pero no verifica nada. En matemáticas eso es peligroso: produce demostraciones con apariencia impecable que son falsas, o citas a documentos que no dicen lo que el modelo afirma. A eso se le llama *alucinación*.

En este curso se han estudiado las nociones básicas de Isabelle que permite verificar si la prueba de un teorema descrita en un texto es correcta y en caso contrario donde falla. El proyecto junta las dos cosas: un modelo de lenguaje que redacta y busca, y un asistente de pruebas que juzga.

El proyecto tiene dos mitades que se califican por separado:

1. **El asistente**: recuperación de información, modelo de lenguaje y verificación con Isabelle.
2. **Una formalización propia**: un pequeño desarrollo en Isabelle, no visto en clase, con ejercicios y soluciones verificadas. Con esos ejercicios se evaluará los diferentes asistentes propuestos por los estudiantes.

2. Qué debe hacer el asistente

R1. Responder con fuentes. Contesta en español preguntas conceptuales (deducción natural, inducción, Isar, automatización) y **cita** la parte del texto en que se apoya: documento y sección, página o nombre del lema.

R2. Admitir lo que no sabe. Si la literatura consultada no respalda una respuesta, lo dice. Puede responder de conocimiento general solo si lo advierte. Una cita inventada es el peor error posible.

R3. Demostrar con verificación. Dado un enunciado en Isabelle, propone una demostración y la verifica con Isabelle. Si falla, le devuelve al modelo el error de Isabelle y reintenta, hasta un número máximo de intentos. Siempre muestra el veredicto. **Nunca presenta como correcta una demostración que Isabelle no aceptó.**

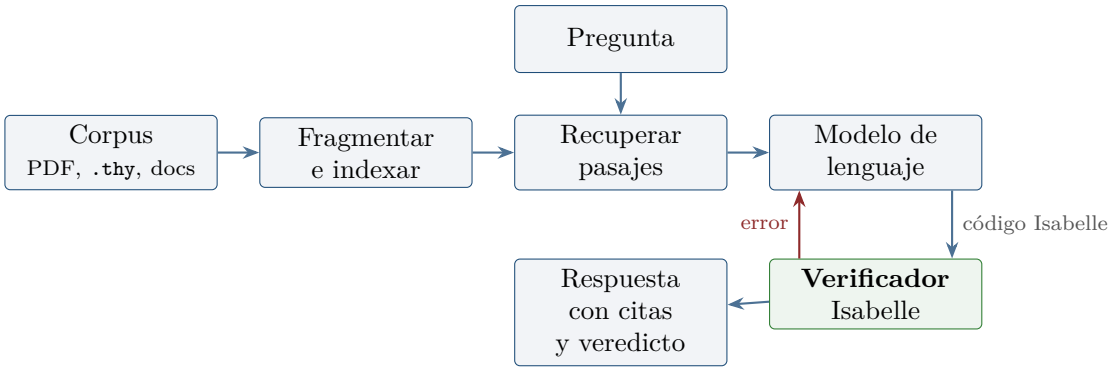
R4. Respetar el estilo. Si se pide deducción natural, usa solo reglas (`rule`, `erule`, `..`, `assumption`...) y no métodos automáticos como `auto`, `simp` o `blast`.

R5. Dejar registro. Por cada respuesta guarda los pasajes recuperados, lo que se le pidió al modelo, cada intento y cada veredicto de Isabelle. Sin registros no se califica.

R6. (Avanzado) Refutar antes de demostrar. Ante un enunciado falso, lo reconoce (con `quickcheck`, `nitpick` o un contraejemplo) en vez de insistir en demostrarlo.

3. Arquitectura sugerida

No es obligatoria, pero es el camino corto:



La flecha roja es el corazón del proyecto: el error de Isabelle vuelve al modelo, que corrige y lo intenta de nuevo.

4. Conjunto de datos

Material	Qué es	
Notas de Alonso (2013) y su libro (2018)	Texto base del curso, con ejercicios resueltos	
Teorías del curso	`isabelle/`: las notas portadas a Isabelle2025-2 y las teorías de ejemplos	Del curso
Enunciados de ejercicios	`ejercicios/`: sin soluciones	Del curso
Diapositivas del curso	Temas 1 a 8 y ejemplos	Del curso
Documentación de Isabelle	`prog-prove`, `isar-ref`, `sledgehammer`, `functions`, `datatypes`... Viene con la instalación, en la carpeta `doc`	BSD

- **Fragmentar por lemas, no por tamaño.** Un lema partido por la mitad no le sirve a nadie. Las teorías `.thy` se fragmentan por comandos: un `lemma` con su demostración, una `definition`, un `fun` con sus ecuaciones.
- **Los símbolos tienen dos formas.** En los archivos se escribe `\<and>`; en los PDF y en las preguntas, ∧. Hay que normalizar para que la búsqueda encuentre las dos igual. El verificador ya convierte en el sentido que necesita Isabelle.

5. Recursos

Reciben la carpeta `recursos/`, en Python sin dependencias externas:

`verificador.py`	`verificar(texto)` le pregunta a Isabelle y devuelve si la acepta, los errores (limpios y con la línea del texto enviado) y el tiempo. Convierte los símbolos Unicode, corta las pruebas que no terminan y no deja pasar `sorry`.
`benchmark_R1.json`	Benchmark público: los 58 ejercicios de la relación 1, estilo deducción natural.
`evaluar.py`	Califica un archivo de respuestas contra un benchmark. Es el mismo que usará el profesor.
`construir_benchmark.py`	Convierte una relación `.thy` en un benchmark.
`comparar_enunciados.py`	Comprueba que enunciados y soluciones enuncian lo mismo.

El `README.md` de los recursos explica cómo usarlos. Una verificación tarda unos 4 o 5 segundos.

6. Cómo se evalúa el asistente

6.1 Benchmarks de demostración

Su asistente recibe un archivo de preguntas y produce un archivo de respuestas: para cada pregunta, la demostración que va después del enunciado. `evaluar.py` la verifica con Isabelle y la clasifica:

`aceptada`	Isabelle la acepta y respeta el estilo pedido
`estilo`	Isabelle la acepta, pero se pedía deducción natural y usa métodos automáticos
`rechazada`	Isabelle no la acepta
`trampa`	intenta añadir lemas, axiomas o `sorry`: ni se verifica

- **Público** (R1, 58 ejercicios). Lo tienen desde el principio. Sus soluciones están en el libro de 2018, que es parte del conjunto de datos: mide sobre todo que la recuperación funcione.
- **Privado**. Se arma al final con ejercicios cuyas soluciones *no* están en el conjunto de datos: los de la relación de la Parte II y los que aporten los demás grupos de estudiantes. Reciben las preguntas, su asistente las responde, y tienen un plazo corto para entregar. Mide razonamiento y verificación.

6.2 Preguntas conceptuales

Se tiene un banco privado de preguntas como las del anexo A. Cada respuesta se califica de 0 a 3: 3 si es correcta y con citas reales, 0 si es incorrecta *o tiene una cita inventada*. Algunas preguntas prueban comportamiento: si el asistente admite no saber, si detecta una premisa falsa, si declina lo que no es de su tema.

6.3 Comparación con la línea base

El informe debe mostrar qué aporta cada pieza, corriendo el benchmark público en tres configuraciones con el *mismo* modelo:

1. solo el modelo, sin recuperación y sin verificación;
2. con recuperación, sin verificación;
3. el asistente completo.

Se califica la mejora, no el modelo

Un grupo con un modelo de pago y uno con un modelo local pequeño no van a tener el mismo porcentaje. Por eso se califica la mejora sobre la línea base del mismo modelo, no el número absoluto. Y por encima de todo, la honestidad: un asistente que resuelve el 30 % y nunca miente sobre el veredicto vale más que uno que resuelve el 60 % y a veces presenta pruebas sin verificar.

7. Actividades a desarrollar por cada grupo

Cada grupo escribe un pequeño desarrollo en Isabelle sobre un tema no visto en clase, en el formato de las teorías vistas en el curso:

- un archivo de **enunciados**, con al menos 8 ejercicios terminados en `oops`, de dificultad variada (al menos dos `[***]`);
- un archivo de **soluciones**, con los mismos nombres de lema y todo demostrado;
- un informe corto que enlace cada paso del argumento informal con el lema formal correspondiente.

`comparar_enunciados.py` debe dar que los dos archivos coinciden. Ideas de tema: ordenamiento por inserción y su corrección; árboles binarios (espejo, tamaño, recorridos); factorial, Fibonacci o potencias y sus propiedades; un evaluador de expresiones aritméticas; propiedades de relaciones (reflexiva, simétrica, transitiva).

Estos ejercicios entran al benchmark privado: **con ellos se evalúa el asistente de los otros equipos**. Ejercicios demasiado fáciles no ayudan a nadie a distinguirse; ejercicios imposibles, tampoco.

8. Grupos, y entregables

Grupos de máximo tres estudiantes.

Entrega final

1. **Repositorio** con el código y un `README` que permita instalarlo y correrlo desde cero.
2. **Respuestas** a los benchmarks público y privado, con sus registros.
3. **Informe técnico**: arquitectura y decisiones; resultados y comparación con la línea base; análisis de errores con *ejemplos reales* de demostraciones falsas que Isabelle atrapó.
4. **Formalización**: enunciados, soluciones e informe informal–formal.

Se realizara una sesión para que cada grupo sustente su trabajo.

9. Calificación

En la evaluación se tendrán en cuenta los siguientes aspectos.

Componente	Peso
Formalización : compila, enunciados y soluciones coinciden, calidad de los ejercicios, informe informal–formal	
Verificación integrada: bucle generar–verificar, veredictos honestos, registros	
Recuperación: corpus, fragmentación, citas reales	
Resultados: benchmarks y preguntas conceptuales, medidos como mejora sobre la línea base	
Informe técnico: comparación con la línea base y análisis de errores	

10. Reglas

- **Cualquier modelo.** Por API o local. El sistema debe permitir cambiar de modelo desde la configuración. No se exige pagar nada: todo el proyecto puede hacerse con modelos gratuitos o locales.
- **Honestidad del asistente.** Decir «verificado» sin que Isabelle lo haya verificado, o una respuesta clasificada como `trampa`, anula el componente de verificación.
- **Las respuestas las produce el asistente.** Responder a mano el benchmark privado es copia. Los registros y la demostración en vivo existen para comprobarlo.
- **Uso de IA para programar.** Permitido. Declárenlo en el informe: qué herramienta y para qué.
- **Fuentes.** El asistente cita; el informe también. Nada de datos personales en el corpus.

A. Ejemplos de preguntas conceptuales

Representativas del tipo de pregunta, no las del banco privado.

1. ¿Qué dice la regla `mp` y en qué se diferencia de `impI`?
2. ¿Qué significan los dos puntos `..` al final de un paso de Isar?
3. ¿Qué diferencia hay entre `have` y `show`?
4. ¿Para qué sirve `obtain`? Dé un ejemplo con un existencial.
5. Explique la inducción sobre listas: ¿qué hay que demostrar en cada caso?
6. ¿Qué es `assms(2)`?
7. Demuestre en Isabelle, solo con deducción natural, que $p \wedge q \vdash q \wedge p$.
8. Demuestre que `longitud (conc xs ys) = longitud xs + longitud ys`.

B. Formato de los archivos

Benchmark (lo produce `construir_benchmark.py`): una lista de preguntas con `id`, `estilo` (`dn` o `libre`), `enunciado` y `contexto`, que es todo lo que va antes del enunciado en la teoría: definiciones y lemas previos. Su asistente necesita el contexto para entender el enunciado.

Respuestas (lo produce su asistente):

```
{
 "R1/ejercicio_1": "proof -\n  show \"q\" using 1 2 by (rule mp)\nqed",
 "R1/ejercicio_2": "using assms by auto"
}
```

Con ese archivo, la primera sale `aceptada` y la segunda `estilo`: es correcta, pero usa `auto` donde se pedía deducción natural.